On-Device Continual Personalization for RSS Readers: A TinyML Approach to Privacy-Preserving Recommendation

May 15, 2026

Abstract

This draft proposes an on-device recommender framework for RSS reading that performs continual personalization using only local user behavior. The objective is to mitigate model collapse and privacy risks by keeping interaction logs, feature extraction, adaptation, and model updates entirely on-device. The work targets Android first and is designed to remain compatible with future TinyML transfer constraints for STM32-class hardware. The current implementation path prioritizes a deployable local baseline with strict efficiency constraints on latency, memory footprint, storage growth, and battery cost.

Contents

1 Introduction

1.1 Problem Statement

RSS recommendation in mobile reading applications must adapt to changing user interests while respecting strict privacy and efficiency constraints. The practical problem is that useful personalization requires frequent behavioral feedback, yet exporting detailed reading traces to remote servers introduces privacy risk and weakens deployability in bandwidth-limited or offline settings. The scientific problem is therefore to determine whether a lightweight recommender can learn continuously from on-device interactions, remain responsive under mobile resource budgets, and preserve acceptable ranking quality under concept drift.

1.2 Motivation

Local-only continual learning is motivated by three constraints. First, RSS reading behavior is highly non-stationary: short-term interests can change rapidly across topics, languages, and reading contexts. Second, cloud-first recommenders typically assume centralized logging, delayed retraining, and persistent connectivity, all of which conflict with privacy-preserving mobile use. Third, app-level deployment requires bounded storage, update cost, and battery impact rather than only offline ranking accuracy. For this reason, the present work prioritizes a fully local adaptation loop before introducing heavier model classes.

1.3 Contributions

Current contributions of the implementation-oriented phase are as follows:

- A deployable Android RSS reader with fully local interaction capture for impressions, opens, read sessions, search, and stage transitions.
- A lightweight continual learner that starts from an empty user profile, updates topic weights incrementally, detects drift using short-vs-long exponential moving averages, and feeds those signals back into ranking.
- A deployment-first research protocol that treats latency, storage growth, battery cost, and future STM32 transferability as first-class evaluation criteria.

2 Background and State of the Art

2.1 On-Device Recommender Systems

TODO: Summarize key approaches from the DeviceRS survey.

2.2 Continual Learning at the Edge

TODO: Summarize edge continual training assumptions and limits.

2.3 TinyML Online Learning

TODO: Extract STM32-relevant constraints and lessons from TinyOL.

2.4 Privacy-by-Design in Mobile Systems

TODO: Map privacy principles to this architecture.

3 Research Gap and Novelty

3.1 Gaps in Related Work

The related-work matrix highlights a consistent deployment gap. Existing work discusses on-device recommendation, edge continual learning, TinyML online adaptation, and privacy-by-design, but these strands are rarely integrated into a single app-embedded personalization loop with explicit runtime budgets. Prior work also tends to emphasize either conceptual taxonomy, microcontroller feasibility, or privacy architecture in isolation. What remains underexplored is a reproducible mobile recommender that learns locally from real user behavior, exposes measurable storage and update overhead, and preserves a clean transfer path to smaller embedded targets.

3.2 Proposed Novelty

The novelty claim of this work is not a new large-scale recommender architecture. Instead, it is the joint design of a privacy-preserving, drift-aware, deployment-constrained continual personalization loop embedded inside an RSS reader. The boundary condition is explicit: the current phase uses lightweight topic-level state and starts from scratch per user rather than relying on a pretrained global model. More expressive neural and quantized architectures remain future phases once the local baseline is validated empirically.

3.3 Deployment-Centric Claims

The deployment-centric claims of the project are defined by measurable thresholds rather than accuracy alone:

- Feed ranking and re-ranking should remain interactive, with target inference cost below 50ms per refresh on a commodity smartphone.
- Online updates should remain lightweight enough for event-driven execution, with target update latency below 100ms per incremental learning step.
- Persistent recommender state should remain compact enough for long-running local use, with bounded event retention and sub-megabyte model-state overhead in the baseline phase.
- Privacy guarantees should hold without exporting raw behavior traces or model updates off-device during the baseline study.

4 Methodology

4.1 System Architecture

TODO: Add architecture figure and component breakdown.

4.2 User Behavior Model

Based on empirical observation of real user interactions with RSS readers, we identify three distinct engagement stages:

- **Stage 1 (Feed Tab):** Browsing list of articles with filtering (all/read/unread), sorting (date), search, and implicit scroll-based item exposure. Users spend time proportional to subject interest, title characteristics, and content language.

- **Stage 2 (Article Preview):** Decision point where user views article snippet (title, image, description) and chooses an action: open in browser, read in app, save for later, share, or return.
- **Stage 3 (Reader):** Full article consumption with available features: translate, read-aloud, notes, bookmark, share, and save. Reading speed and dwell time correlate with character count, language, and content quality.

Behavioral signals extracted at each stage inform the continual learner. Stage transitions and feature usage (e.g., translation, read-aloud, annotations) serve as learning labels and engagement proxies.

4.3 Signal Extraction Schema

- **Impressions:** Article rank, filter, sort order, title length, image presence, detected language.
- **Opens:** Action type (preview, read-in-app, browser), article metadata.
- **Read sessions:** Dwell time, max scroll depth, feature usage (translate, read-aloud, notes), content language and length.
- **Search:** Query string for intent inference (future work).
- **Stage transitions:** Entry/exit timestamps and dwell per stage.

4.4 Efficiency Cascade: Joint Optimization of Architecture, Quantization, and Learning

The research strategy centers on a three-tier efficiency pipeline designed to make local continual learning feasible on resource-constrained mobile and embedded platforms:

1. **Architecture Search (µNAS):** Automated discovery of minimal-footprint models under strict phone/microcontroller latency and memory budgets. Reframe RSS recommendation as lightweight decision operations (topic scoring, ranking) rather than end-to-end black-box models.
2. **Weight Quantization (BNN):** Binarization of learned weights and activations reduces model size by 8-32x and speeds inference on low-power hardware. Evaluate utility loss (ranking quality) versus compression gain (storage, latency, power).
3. **Continual Learning (CL + TinyOL):** On-device incremental adaptation of the quantized model using bounded replay and drift-aware scheduling. Target < 100ms update latency per user event.

Expected Outcome: Smartphone recommendation model below 5MB, inference below 50ms, update below 100ms, with validated transfer to STM32 microcontroller.

4.5 Selected Implementation Approach (Current)

- **Phase A (current):** Lightweight local continual learner with event-driven updates and bounded replay, fully on-device.
- **Phase B (near-term):** Constrained architecture search (µNAS-inspired) combined with binarized weight quantization (BNN) to reduce model footprint 8-32x. Validate on-device inference latency and update cost under phone constraints.
- **Phase C (research goal):** TinyML transfer track with compact model/state representation aligned with STM32 constraints. Cross-device evaluation to validate generalization.

4.6 Local Interaction Pipeline

The local interaction pipeline is currently implemented as an event log with bounded replay capacity and explicit retention controls.

- **Event families:** `impression`, `open`, `read_session`, `search`, and `stage_transition`.
- **Session enrichment:** Reader sessions store dwell time, max scroll depth, completion signal, and feature usage flags (translation, read-aloud, note-taking).
- **Bounded storage:** Event buffer is capped to 5000 records to prevent unbounded growth.
- **Retention policy:** Time-based pruning is supported (default 30 days; configurable to longer windows in settings).
- **Cursor-based replay:** Continual updates process only unseen events via persistent cursor, reducing repeated compute overhead.

4.7 Continual Learning Strategy

The baseline learner is an event-driven additive updater over topic-specific preference weights.

- **Signal-to-update mapping:** opens provide positive intent signal; read sessions add dwell-depth-completion weighted reinforcement.
- **Replay model:** Incremental cursor replay applies updates only to newly recorded events since last step.
- **Stability guardrails:** Topic weights are clamped to bounded range to avoid runaway growth and collapse.
- **Drift detector (implemented):** For each topic, short-window and long-window EMAs of update deltas are tracked; divergence is computed as $|EMA_{short} - EMA_{long}|$.
- **Drift alerting:** Topics whose divergence exceeds threshold after minimum evidence are marked as drift-flagged and receive temporary priority boost in the feed ranking policy, with scheduler integration remaining future work.

4.8 Efficiency and Footprint Constraints

Current budget targets are fixed, while observed values remain to be measured in the pilot study:

- **Inference latency target:** below 50ms per feed refresh under typical list sizes.
- **Update latency target:** below 100ms per incremental learning step triggered by new interactions.
- **Storage target:** bounded event store (currently 5000 records) and compact topic-level model state.
- **Observed-values TODO:** add measured latency, storage growth, battery drain, and thermal observations from controlled pilot sessions.

4.9 Privacy Model

The baseline privacy model assumes a semi-honest device environment but no trusted remote personalization service. Interaction events, topic weights, drift statistics, and retention controls remain local to the device through app-managed storage. The main privacy guarantee is therefore zero intentional export of raw behavioral traces for training. Remaining threats include device compromise, local backup leakage, and future telemetry additions; these must be handled explicitly before any broader deployment study.

5 Experimental Protocol

5.1 Research Questions and Hypotheses

The current draft is organized around four core questions:

- **RQ1:** Can a local continual learner that starts from scratch improve ranking utility relative to non-personalized baselines during real RSS use?
- **RQ2:** Does drift-aware ranking improve adaptation speed when user interests shift across topics?
- **RQ3:** Can these gains be achieved under strict mobile latency, storage, and battery constraints?
- **RQ4:** Does a strict local-only pipeline remain practical without degrading user experience or requiring cloud-side coordination?

Expected hypotheses are H_1: continual local adaptation outperforms static ordering, H_2: drift-aware prioritization shortens recovery after preference shifts, and H_3: the lightweight baseline remains within the stated deployment budgets.

5.2 Datasets and Real-Usage Collection

The system is already suitable for pilot-scale real-usage collection. When on-device learning is enabled, the application records local interaction events in real time and updates the user model incrementally. No pretrained model is required for this phase; each user can begin from an empty topic-weight profile and accumulate personalized state through normal reading behavior. The intended protocol is a staged deployment: first, single-user longitudinal testing to validate logging and update stability; second, a small pilot with consenting users over multiple weeks; third, controlled drift scenarios to measure recovery speed. A remaining TODO is to define the exact participant count, study duration, and feed selection policy for the first formal experiment.

5.3 Baselines and Ablations

The baseline and ablation plan should separate personalization gains from architectural complexity:

- Reverse-chronological ordering with no personalization.
- Topic-weight ranking without continual updates after initialization.
- Continual learner without drift-based boosts.
- Continual learner with drift-based boosts enabled.
- Future ablation: warm-start initialization from a pretrained prior versus pure from-scratch learning.

5.4 Metrics

Primary metrics should include feed open-through rate, read-session completion rate, dwell-weighted engagement, adaptation delay after detected drift, ranking freshness, storage growth, update latency, and battery cost. Because this is a deployability-focused study, every utility metric should be paired with an efficiency metric rather than reported alone.

5.5 Observability and Monitoring

Current monitoring exists at two levels. First, the app persists local event logs, model state, and drift statistics in device storage. Second, the settings screen exposes a summary view containing recent event counts, top learned topics, and strongest drift signals, while runtime failures are surfaced through local console warnings and errors during development. A concrete TODO for the next revision is to export structured experiment snapshots (for example, anonymized local summaries, latency traces, and battery observations) so that pilot results can be analyzed reproducibly without uploading raw interaction histories.

5.6 Primary Evaluation Targets

- Accuracy and ranking quality under non-stationary user behavior.
- Adaptation speed after concept drift events.
- Resource efficiency: latency, memory footprint, storage overhead, energy cost.
- Privacy robustness under local-only constraints and no raw behavior export.

6 Preliminary Implementation Status

- v2.0.4: Integrated learned topic weights and drift-flagged topic boosts into feed ranking when on-device learning is enabled.
- v2.0.3: Added on-device drift detector using short-vs-long EMA divergence per topic with thresholded topic-level flags.
- v2.0.2: Enhanced event schema to capture stage transitions, feature usage (translate, read-aloud, notes), and article metadata (language, title length, content length).
- v2.0.2: Integrated user behavior model from empirical observation into signal extraction.
- v2.0.2: Added strategy for efficiency cascade: µNAS (architecture search) + BNN (weight quantization) + CL (continual learning).
- v2.0.1: Added local on-device interaction event pipeline in app code.
- v2.0.1: Added feed-level impression/open logging and reader dwell-depth session logging.
- v2.0.1: Added settings controls for local learning enablement, retention, summary, and reset.

7 Threats to Validity

The main internal-validity risk is that early gains may reflect interface recency or feed composition rather than genuine personalization. External validity is limited because the present implementation targets one RSS-reader workflow and one mobile stack. Construct validity depends on whether opens, dwell time, and scroll depth are adequate proxies for utility. Conclusion validity will remain weak until the pilot includes enough users and sufficiently long sessions to characterize drift, stability, and resource cost under realistic behavior.

8 Ethics and Privacy Considerations

All user studies should require explicit consent, a visible explanation that personalization occurs locally, and an in-app reset path that clears behavioral history and model state. Because the pipeline is designed to avoid remote export of raw traces, ethical emphasis shifts toward transparency, retention limits, and ensuring that optional experiment summaries do not re-identify users. A remaining TODO is to specify the consent screen wording and any institutional review requirements for the first pilot.

9 Conclusion and Next Steps

The project has reached the point where real-time local data collection and from-scratch continual adaptation are feasible inside the app. The next technical milestone is not C or microcontroller deployment yet; it is a monitored pilot that measures whether the current JavaScript-based baseline improves ranking quality under real use while staying within mobile resource budgets. Immediate next steps are to add structured observability for latency and battery, define the pilot protocol, and then use those findings to justify the later transition toward µNAS, BNN, and STM32-oriented implementation constraints.